تمرین چهارم درس طراحی سیستم‌های دیجیتال

دانشگاه صنعتی شریف – نیمسال دوم ۰۲-۰۳

استاد: دکتر فصحتی
دانشجو: روژین تقی‌زادگان
شماره دانشجویی: ۴۰۱۱۰۵۷۷۵



فهرست مطالب

ASM Chart 2

منابع لازم برای Datapath 3

MUX 3

register 4

adder 5

signed comparator 6

unsigned comparator (abs) 7

n-bit OR 8

SR Flip-Flop 9

Datapath 10

نمودار حالت 12

Control Unit 13

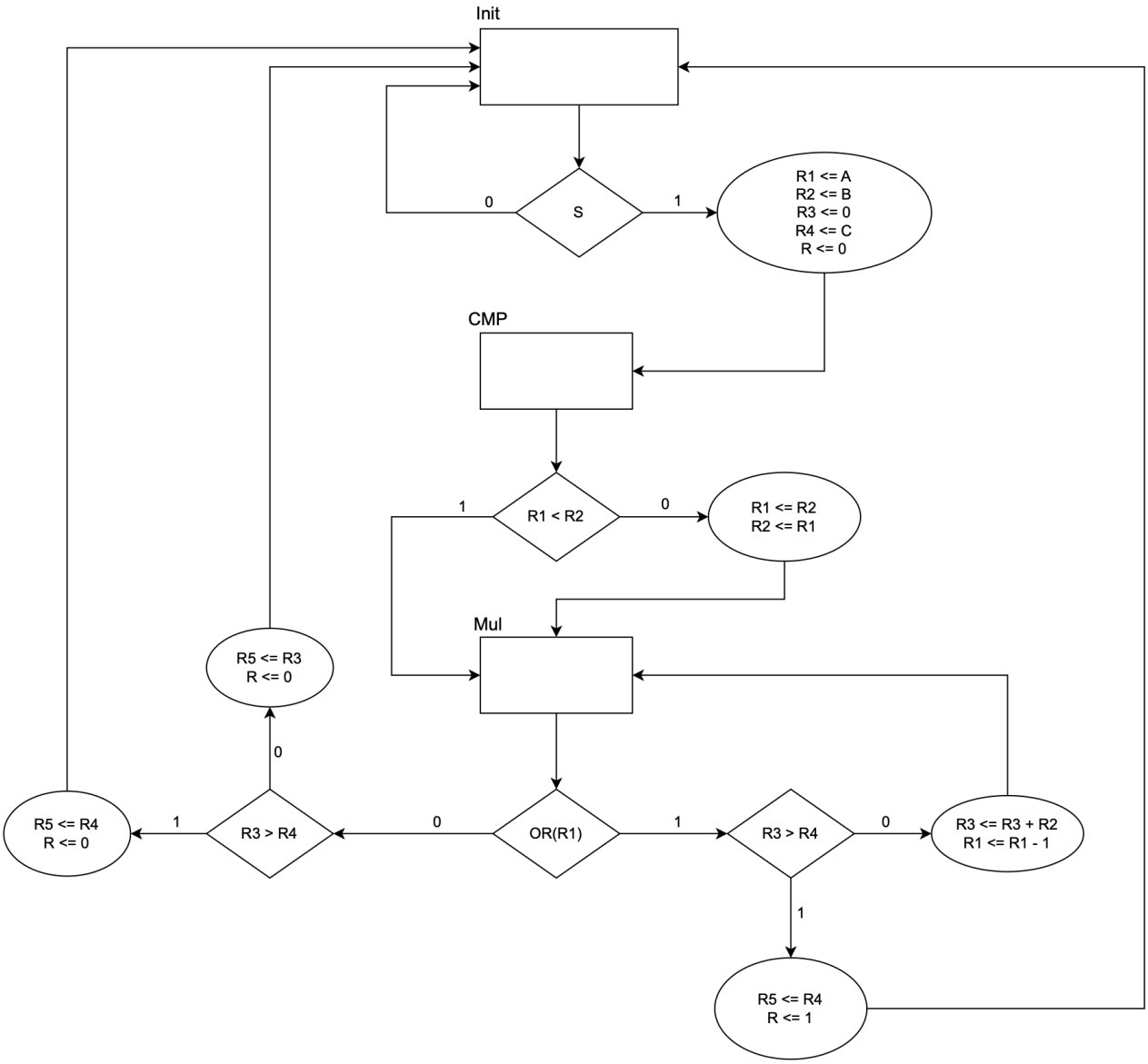
اتصال datapath و control unit 15

Test Bench 16

در این تمرین قصد داریم مداری طراحی کنیم که سه عدد A و B و C را ورودی می‌گیرد و با فشردن دکمه start، دو عدد A و B را با روش جمع متوالی در هم ضرب می‌کند و در نهایت مینیمم AB و C را خروجی می‌دهد.

ASM Chart

ASM Chart مدار خواسته شده به شکل زیر است:



مقایسه‌ای که در حالت CMP انجام می‌شود مقایسه قدر مطلق‌ها (unsigned) است تا بتوان کوچک‌ترین مقدار را در R1 قرار داد که بتوان عملیات ضرب را در کمترین کلاک ممکن انجام داد.

مقایسه‌ای که بین R3 و R4 در حالت Mul انجام می‌شود مقایسه اعداد با علامتشان است تا بتوان مینیمم حاصل‌ضرب و ورودی C را پیدا کرد.

پس از هر مرحله عملیات جمع در پروسه بدست آوردن حاصل ضرب، یک بار R3 و R4 با هم مقایسه می‌شوند تا در صورتی که حاصل ضرب از ورودی C بیشتر شد، عملیات محاسبه حاصل ضرب متوقف شود و ورودی C در خروجی قرار گیرد.

دلیلی که پس از اینکه OR(R1) صفر شد دوباره R3 و R4 را مقایسه می‌کنیم برای هندل کردن edge case است. زمانی که از ابتدا در R1 مقدار صفر قرار گیرد، مقدار OR(R1) از ابتدا صفر می‌شود. بنابراین لازم است یک بار R3 و R4 مقایسه شود.

منابع لازم برای Datapath

با توجه به ASM Chart به منابع زیر برای datapath احتیاج داریم:

- 3 x MUX
- 5 x register
- 1 x adder
- 1 x signed comparator
- 1 x unsigned comparator (abs)
- 1 x n-bit OR
- 1 x SR Flip-Flop

MUX

```
`define width 32

module MUX (in0, in1, select, out);

parameter width = `width;
input wire [width-1:0] in0, in1;
input wire select;
output wire [width-1:0] out;

assign out = select ? in1 : in0;

endmodule
```

ماژول مربوطه

```
`define width 32

module MUX_TB ();

parameter width = `width;
reg [width-1:0] in0, in1;
reg select;
wire [width-1:0] out;

MUX mux1 (in0, in1, select, out);

initial begin
    in0 = 32'h81290734;
    in1 = 32'h01627903;

    select = 1;
    #5 $display("out = %h", out);
    select = 0;
    #5 $display("out = %h", out);
end

endmodule
```

Test Bench

```
out = 01627903
out = 81290734
```

خروجی تست

register

رجیستر ساخته شده دارای ۴ سیگنال برای لود، ریست، decrement و increment است. عملیات که با فعال شدن هر سیگنال انجام می‌شود در ماژول مربوطه مشخص شده است.

```
`define width 32

module register (clk, rst, ld, inc, dec, in, out);

parameter width = `width;
input wire clk, rst, ld, inc, dec;
input wire signed [width-1:0] in;
output reg signed [width-1:0] out;

always @(posedge clk) begin
    if (rst)
        out <= 0;
    else if (ld)
        out <= in;
    else if (inc)
        out <= out+1;
    else if (dec)
        out <= out-1;
end

endmodule
```

ماژول مربوطه

```
`define width 32

module register_TB ();

parameter width = `width;
reg clk = 0, rst = 0, ld = 0, inc = 0, dec = 0;
reg [width-1:0] in;
wire [width-1:0] out;

register register1 (clk, rst, ld, inc, dec, in, out);

always
    #5 clk <= ~clk;

initial begin
    in = 32'd1834;

    ld = 1;
    #10 $display($time, "\tout = %d", out);
    ld = 0;

    inc = 1;
    #10 $display($time, "\tout = %d", out);
    inc = 0;

    rst = 1;
    #10 $display($time, "\tout = %d", out);
    rst = 0;

    in = 32'd9267;

    ld = 1;
    #10 $display($time, "\tout = %d", out);
    ld = 0;

    dec = 1;
    #10 $display($time, "\tout = %d", out);
    dec = 0;
    $finish;
end

endmodule
```

Test Bench

```
10      out =       1834
20      out =       1835
30      out =          0
40      out =       9267
50      out =       9266
```

خروجی تست

adder

```
`define width 32

module adder (in1, in2, sign, sum);

parameter width = `width;
input wire signed [width-1:0] in1, in2;
input wire sign;
output wire signed [width-1:0] sum;
assign sum = sign ? in1 - in2 : (in1 + in2);

endmodule
```

ماژول مربوطه

```
sum =         59
sum =         75
sum =        -10
sum =          8
```

خروجی تست

این ماژول هم می‌تواند حاصل جمع دو ورودی و هم حاصل تفریق دو ورودی را محاسبه کند. تعیین این که کدام عمل را انجام دهد بر عهده بیت sign است. در صورتی که بیت sign یک باشد، حاصل تفریق و در صورتی که صفر باشد حاصل جمع دو ورودی محاسبه می‌شود.

علت افزودن این قابلیت به جمع کننده این است که در صورتی که شمارنده (R1) عدد منفی باشد، لازم است در هر مرحله از رجیستر R3 که حاصل‌ضرب در آن محاسبه می‌شود به اندازه R2 کم شود (به جای آنکه جمع شود)

در واقع در صورتی که علامت R1 مثبت باشد لازم است R2 به مقدار حاصل اضافه شود و در صورتی که علامت R1 منفی باشد لازم است R2 از مقدار حاصل کم شود. بنابراین ورودی ماژول یعنی بیت sign در واقع بیت علامت R1 است.

```
`define width 32

module adder_TB();

parameter width = `width;
reg signed [width-1:0] in1, in2;
reg sign;
wire signed [width-1:0] sum;

adder adder1(in1, in2, sign, sum);

initial begin
    in1 = 32'd67;
    in2 = 32'd8;
    sign = 1;
    #5 $display("sum = %d", sum);

    in1 = 32'd67;
    in2 = 32'd8;
    sign = 0;
    #5 $display("sum = %d", sum);

    in1 = 32'hFFFFFFFF;
    in2 = 32'd9;
    sign = 1;
    #5 $display("sum = %d", sum);

    in1 = 32'hFFFFFFFF;
    in2 = 32'd9;
    sign = 0;
    #5 $display("sum = %d", sum);
end

endmodule
```

Test Bench

signed comparator

```
L:1     E:0     G:0
L:0     E:1     G:0
L:0     E:0     G:1
L:0     E:0     G:1
```

خروجی تست

این ماژول، اعداد را بر حسب علامتشان مقایسه می‌کند. اعداد مثبت بزرگتر از صفر و صفر بزرگتر از اعداد منفی است.

```
`define width 32

module comparator_s (A, B, L, E, G);

parameter width = `width;
input wire signed [width-1:0] A, B;
output wire L, E, G;

assign L = A < B;
assign E = A == B;
assign G = A > B;

endmodule
```

ماژول مربوطه

```
`define width 32

module comparator_s_TB ();

parameter width = `width;
reg signed [width-1:0] A, B;
wire L, E, G;

comparator_s comparator_s_1 (A, B, L, E, G);

initial begin
    A = 32'd28;
    B = 32'd103;
    #5 $display("L:%b    E:%b    G:%b", L, E, G);

    A = 32'd87;
    B = 32'd87;
    #5 $display("L:%b    E:%b    G:%b", L, E, G);

    A = 32'h00000234;
    B = 32'h80000457;
    #5 $display("L:%b    E:%b    G:%b", L, E, G);

    A = 32'h00000034;
    B = 32'hfffffffc1;
    #5 $display("L:%b    E:%b    G:%b", L, E, G);
end

endmodule
```

Test Bench

unsigned comparator (abs)

```
L:1     E:0     G:0
L:0     E:1     G:0
L:1     E:0     G:0
L:1     E:0     G:0
```

خروجی تست

این ماژول، قدر مطلق اعداد را با هم مقایسه می‌کند. یعنی در صورتی که بیت علامت عدد منفی باشد، آن را مثبت می‌کند و سپس عمل مقایسه را انجام می‌دهد.

```v
`define width 32

module comparator_abs (A, B, L, E, G);

parameter width = `width;
input wire clk;
input wire [width-1:0] A, B;
output wire L, E, G;

wire [width-1:0] A_abs, B_abs;
assign A_abs = A[width-1] ? -A : A;
assign B_abs = B[width-1] ? -B : B;

assign L = A_abs < B_abs;
assign E = A_abs == B_abs;
assign G = A_abs > B_abs;

endmodule
```

ماژول مربوطه

```v
`define width 32

module comparator_abs_TB ();

parameter width = `width;
reg [width-1:0] A, B;
wire L, E, G;

comparator_abs comparator_abs_1 (A, B, L, E, G);

initial begin
    A = 32'd28;
    B = 32'd103;
    #5 $display("L:%b    E:%b    G:%b", L, E, G);

    A = 32'd87;
    B = 32'd87;
    #5 $display("L:%b    E:%b    G:%b", L, E, G);

    A = 32'h00000234;
    B = 32'h80000457;
    #5 $display("L:%b    E:%b    G:%b", L, E, G);

    A = 32'h00000034;
    B = 32'hfffffffc1;
    #5 $display("L:%b    E:%b    G:%b", L, E, G);
end

endmodule
```

Test Bench

n-bit OR

```
`define width 32

module OR_nbit (in, out);

parameter width = `width;
input wire [width-1:0] in;
output wire out;

assign out = |in;

endmodule
```

ماژول مربوطه

```
out = 1
out = 1
out = 1
out = 0
```

خروجی تست

```
`define width 32

module OR_TB ();

parameter width = `width;
reg [width-1:0] in;
wire out;
OR_nbit OR_nbit1(in, out);

initial begin
    in = 32'h80192039;
    #5 $display("out = %b", out);

    in = 32'h12345678;
    #5 $display("out = %b", out);

    in = 32'h00129348;
    #5 $display("out = %b", out);

    in = 32'h00000000;
    #5 $display("out = %b", out);
end

endmodule
```

Test Bench

SR Flip-Flop

```
module SR_FF (input clk, S, R, output reg Q, Q_not);

always @(posedge clk) begin
    if (S & ~R) begin
        Q <= 1;
        Q_not <= 0;
    end
    else if (~S & R) begin
        Q <= 0;
        Q_not <= 1;
    end
end

endmodule
```

ماژول مربوطه

```
10      Q:1     Q_not:0
20      Q:1     Q_not:0
30      Q:0     Q_not:1
40      Q:0     Q_not:1
```

خروجی تست

```
module SRFF_TB ();

reg clk = 0, S, R;
wire Q, Q_not;

SR_FF sr_ff1 (clk, S, R, Q, Q_not);

always
    #5 clk <= ~clk;

initial begin
    S = 1;
    R = 0;
    #10 $display($time, "\tQ:%b    Q_not:%b", Q, Q_not);

    S = 0;
    R = 0;
    #10 $display($time, "\tQ:%b    Q_not:%b", Q, Q_not);

    S = 0;
    R = 1;
    #10 $display($time, "\tQ:%b    Q_not:%b", Q, Q_not);

    S = 0;
    R = 0;
    #10 $display($time, "\tQ:%b    Q_not:%b", Q, Q_not);
    $finish;
end

endmodule
```

Test Bench

Datapath

```
`include "MUX.v"
`include "register.v"
`include "OR_nbit.v"
`include "comparator_abs.v"
`include "comparator_s.v"
`include "adder.v"
`include "SR_FF.v"
`define width 32

module DP (clk, A, B, C,
           ld_1, ld_2, ld_3, ld_4, ld_5,
           rst_3, inc_1, dec_1, select_R1, select_R2, select_R5, S_R, R_R,
           SIGN_R1, CMP_R1_R2, OR_R1, CMP_R3_R4, Q, R5_out, R1_out, R2_out);

parameter width = `width;
input signed [width-1:0] A, B, C;
input clk, ld_1, ld_2, ld_3, ld_4, ld_5;
input rst_3, inc_1, dec_1, select_R1, select_R2, select_R5, S_R, R_R;
output SIGN_R1, CMP_R1_R2, OR_R1, CMP_R3_R4, Q;
output signed [width-1:0] R5_out, R1_out, R2_out;

wire signed [width-1:0] R1_out, R2_out, R3_out, R4_out;
wire signed [width-1:0] mux_R1_out, mux_R2_out, mux_R5_out, adder_out;
wire rst_1, rst_2, rst_4, rst_5;
wire dec_2, dec_3, dec_4, dec_5;
wire inc_2, inc_3, inc_4, inc_5;
wire eq_1, gr_1, eq_2, gr_2;
wire Q_not;

assign SIGN_R1 = R1_out[width-1];

MUX mux1(A, R2_out, select_R1, mux_R1_out);
MUX mux2(B, R1_out, select_R2, mux_R2_out);
MUX mux5(R3_out, R4_out, select_R5, mux_R5_out);

register R1(clk, rst_1, ld_1, inc_1, dec_1, mux_R1_out, R1_out);
register R2(clk, rst_2, ld_2, inc_2, dec_2, mux_R2_out, R2_out);
register R3(clk, rst_3, ld_3, inc_3, dec_3, adder_out, R3_out);
register R4(clk, rst_4, ld_4, inc_4, dec_4, C, R4_out);
register R5(clk, rst_5, ld_5, inc_5, dec_5, mux_R5_out, R5_out);

adder adder1(R3_out, R2_out, SIGN_R1, adder_out);
OR_nbit OR_nbit1 (R1_out, OR_R1);
comparator_abs cmp_R1_R2 (R1_out, R2_out, CMP_R1_R2, eq_1, gr_1);
comparator_s cmp_R3_R4 (R3_out, R4_out, CMP_R3_R4, eq_2, gr_2);
SR_FF SR_FF1 (clk, S_R, R_R, Q, Q_not);

endmodule
```

ماژول مربوطه

توضیح Datapath:
در مداری که ساختیم، ثبات R1 برای نگهداری ورودی اول (A)، ثبات R2 برای نگهداری ورودی دوم (B)، ثبات R3 برای نگهداری حاصل جمع متوالی و حاصل‌ضرب نهایی دو ثبات R1 و R2، ثبات R4 برای نگهداری ورودی سوم (C) و ثبات R5 برای نگهداری خروجی نهایی مدار است.

در این ماژول ابتدا ورودی‌ها و خروجی‌های مدار را مشخص می‌کنیم. علاوه بر سه ورودی اصلی (A, B, C)، تمامی سیگنال‌های کنترلی که کنترل‌یونیت آن‌ها را تغییر می‌دهد باید به عنوان ورودی به ماژول داده شوند. (سایر سیگنال‌های دیگر که کنترل یونیت آن‌ها را تغییر نمی‌دهد را به عنوان wire میانی در ادامه ماژول تعریف کرده‌ایم). خروجی‌های ماژول شامل سینگال‌هایی است که کنترل‌یونیت بر اساس آن‌ها حالت بعدی مدار و سیگنال‌های کنترلی مدار برای انجام اعمال معین را تعیین می‌کند. (خروجی‌های datapath ورودی‌های کنترل یونیت هستند)

در ادامه wireهای میانی مدار تعریف شده‌اند (مانند سیگنال reset یا increment یا decrement رجیسترهایی که کنترل‌یونیت آن‌ها را تغییر نمی‌دهد)
در ادامه منابع ساخته شده برای datapath استفاده شده‌اند.

چون در ثبات R1 دو مقدار A یا R2 می‌تواند لود شود، قبل از آن یک MUX می‌گذاریم و این ورودی‌ها را به MUX می‌دهیم و خروجی MUX را به ورودی ثبات R1 وصل می‌کنیم.
چون در ثبات R2 دو مقدار B یا R1 می‌تواند لود شود، قبل از آن یک MUX می‌گذاریم و این ورودی‌ها را به MUX می‌دهیم و خروجی MUX را به ورودی ثبات R2 وصل می‌کنیم.
چون در ثبات R5 دو مقدار R3 یا R4 می‌تواند لود شود، قبل از آن یک MUX می‌گذاریم و این ورودی‌ها را به MUX می‌دهیم و خروجی MUX را به ورودی ثبات R5 وصل می‌کنیم.

برای استفاده از روش جمع متوالی در محاسبه حاصل‌ضرب از جمع‌کننده استفاده می‌کنیم و همانطور که در قبل توضیح داده شده بود، بیت علامت R1 را به عنوان ورودی به جمع‌کننده می‌دهیم تا مشخص کنیم که برای محاسبه حاصل‌ضرب باید عملیات جمع انجام دهیم یا عملیات تفریق.

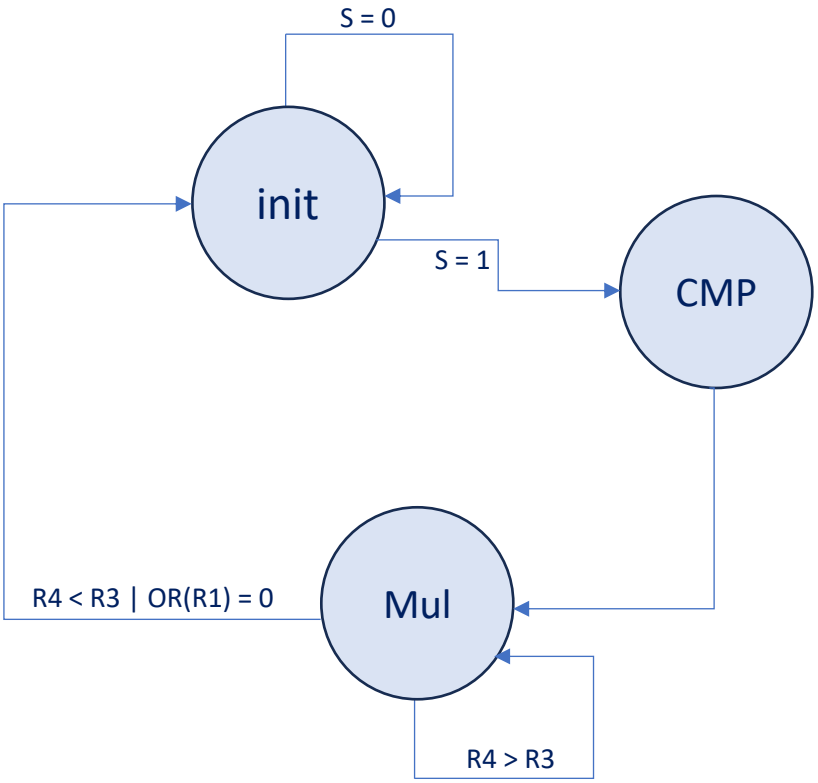
برای بدست آوردن حاصل OR بیت‌های ثبات R1 از ماژول OR_nbit استفاده می‌کنیم.

برای مقایسه دو ثبات R1 و R2 از مقایسه‌کننده بدون علامت (abs) استفاده می‌کنیم تا بتوانیم کمترین عدد (از نظر مقداری) را در رجیستر R1 قرار دهیم تا بتوانیم عملیات ضرب را در کمترین کلاک سایکل انجام دهیم.

اما برای مقایسه دو ثبات R3 و R4 از مقایسه‌کننده با علامت استفاده می‌کنیم تا بتوانیم مینیمم حاصل‌ضرب و ورودی C (با در نظر گرفتن علامت) را بدست آوریم.

برای نگهداری سیگنال ready از یک فلیپ‌فلاپ SR استفاده کردیم تا بتوانیم با تغییر سیگنال‌های S و R مقدار سیگنال ready را کنترل کنیم.

نمودار حالت



Control Unit

```
module CU (input clk, rst, S, Sign_R1, CMP_R1_R2, OR_R1, CMP_R3_R4,
           output reg L_R1, L_R2, L_R3, L_R4, L_R5, R_R3, Inc_R1, Dec_R1, Sel_R1, Sel_R2, Sel_R5, S_R, R_R);

reg [1:0] p_state, n_state;
localparam[1:0] init = 2'b00, cmp = 2'b01, mul = 2'b10;
```

```
always @(p_state or S or CMP_R1_R2 or OR_R1 or CMP_R3_R4) begin:combinational
    n_state = init;
    L_R1 <= 0;
    L_R2 <= 0;
    L_R3 <= 0;
    L_R4 <= 0;
    L_R5 <= 0;
    R_R3 <= 0;
    Inc_R1 <= 0;
    Dec_R1 <= 0;
    Sel_R1 <= 0;
    Sel_R2 <= 0;
    Sel_R5 <= 0;
    S_R <= 0;
    R_R <= 0;
```

```
case (p_state)
    init: begin
        if(S == 1) begin
            n_state = cmp;
            L_R1 <= 1;
            L_R2 <= 1;
            R_R3 <= 1;
            L_R4 <= 1;
            Sel_R1 <= 0;
            Sel_R2 <= 0;
            S_R <= 0;
            R_R <= 1;
        end

        else begin
            n_state = init;
        end
    end

    cmp: begin
        if(CMP_R1_R2 == 0) begin
            L_R1 <= 1;
            L_R2 <= 1;
            Sel_R1 <= 1;
            Sel_R2 <= 1;
        end
        n_state = mul;
    end
```

توضیح Control Unit:

ابتدا ورودی‌ها و خروجی‌های مدار را مشـــخص می‌کنیم. همین‌طور که در قبل گفته شـد، ورودی‌های کنترل‌یونیت سـیگنال‌هایی هسـتند که این واحد بر اســاس آن‌ها حالت بعدی مدار و ســیگنال‌های کنترلی مدار برای انجام اعمال معین را تعیین می‌کند. خروجی مدار هم شــامل ســیگنال‌های کنترلی اســت که منابع موجود در datapath بر اســاس آن‌ها کار خود را انجام می‌دهند.

کنترل یونیت از دو بخش ترکیبی و ترتیبی ســاخته شــده که بخش ترکیبی بر اســاس حالت مدار، ســیگنال‌های کنترلی و حالت بعدی مدار را مشــخص می‌کند. بخش ترتیبی، حالت کنونی مدار را تعیین می‌کند (حالت کنونی را معادل حالت بعدی که در بخش ترکیبی مشخص شـده بود قرار می‌دهد) یا در صورت ریست شدن مدار، حالت کنونی مدار را معادل حالت اولیه مدار (در اینجا init) قرار می‌دهد.

مدار ما از سه حالت تشکیل شده است:

- حالت اولیه (init)
- حالت مقایسه A و B (compare)
- حالت ضرب (mul)

```
    mul: begin
        if(OR_R1 == 0) begin
            if(CMP_R3_R4 == 1)
                Sel_R5 <= 0;
            else
                Sel_R5 <= 1;

            L_R5 <= 1;
            S_R <= 1;
            R_R <= 0;
            n_state = init;
        end

        else begin
            if(CMP_R3_R4 == 0) begin
                L_R5 <= 1;
                Sel_R5 <= 1;
                S_R <= 1;
                R_R <= 0;
                n_state = init;
            end

            else begin
                if(Sign_R1)
                    Inc_R1 <= 1;

                else if(~Sign_R1)
                    Dec_R1 <= 1;
                L_R3 <= 1;
                n_state = mul;
            end
        end
    end
endcase
```

برای افزایش سرعت مدار و این که مدار در کمترین پالس ساعت خروجی را تعیین کند، هر بار قبل از یک مرحله جمع حین عملیات ضرب، حاصل را با ورودی سوم (C) مقایسه می‌کنیم تا اگر در همان میانه راه حاصل‌ضرب از ورودی سوم بیشتر شده بود، ورودی سوم را خروجی دهیم و عملیات ضرب را تا آخر انجام ندهیم. برای این کار از سیگنال CMP_R3_R4 استفاده کردیم که از datapath خارج می‌شود که حاصل مقایسه حاصل‌جمع و ورودی سوم است. در صورتی که این سیگنال صفر باشد، حاصل‌جمع از ورودی سوم بزرگتر شده، بنابراین خروجی مدار برابر با ورودی سوم قرار می‌گیرد، سیگنال ready برابر با یک می‌شود و مدار به حالت اولیه (init) می‌رود.

همان‌طور که در قبل گفته شد، در صورتی که بیت علامت R1 مثبت باشد عملیات جمع و در صورتی که منفی باشد عملیات تفریق انجام می‌شود.
حال برای آن‌که ثبات R1 به صورت شمارنده عمل کند، باید در صورتی که مقدار آن مثبت است، یک واحد از آن کم کرده و آن‌قدر این کار را ادامه دهیم تا به صفر برسد.
اما اگر مقدار آن منفی باشد، باید یک واحد به آن اضافه کنیم و آن‌قدر این کار را ادامه دهیم تا به صفر برسد.

```
always @(posedge clk) begin:sequential
    if(rst) begin
        S_R = 0;
        R_R = 1;
        p_state = init;
    end
    else
        p_state = n_state;
end

endmodule
```

در بخش ترتیبی کنترل‌یونیت، همانطور که گفته شد با لبه مثبت کلاک، حالت کنونی مدار آپدیت می‌شود و در صورتی که سیگنال ریست یک باشد، سیگنال ready صفر می‌شود و حالت کنونی مدار برابر با حالت اولیه (init) می‌شود.

اتصال datapath و control unit

```
`include "DP.v"
`include "CU.v"
`define width 32

module circuit (clk, rst, S, A, B, C, R, out, R1_out, R2_out);

parameter width = `width;
input clk, rst, S;
input signed [width-1:0] A, B, C;
output R;
output signed [width-1:0] out, R1_out, R2_out;

DP datapath (clk, A, B, C,
             L_R1, L_R2, L_R3, L_R4, L_R5,
             R_R3, Inc_R1, Dec_R1, Sel_R1, Sel_R2, Sel_R5, S_R, R_R,
             Sign_R1, CMP_R1_R2, OR_R1, CMP_R3_R4, R, out, R1_out, R2_out);

CU controlUnit (clk, rst, S, Sign_R1, CMP_R1_R2, OR_R1, CMP_R3_R4,
                L_R1, L_R2, L_R3, L_R4, L_R5,
                R_R3, Inc_R1, Dec_R1, Sel_R1, Sel_R2, Sel_R5, S_R, R_R);

endmodule
```

ماژول مربوطه

در این ماژول، datapath و control unit را به هم وصل می‌کنیم. همان‌طور که در قبل گفته شد، خروجی‌های datapath ورودی‌های control unit هستند (control unit بر اساس آن‌ها وضعیت مدار و سیگنال‌های کنترلی را تعیین می‌کند) همچنین خروجی‌های کنترل یونیت (سیگنال‌های کنترلی) ورودی‌های datapath هستند.

Test Bench

```
`define width 32

module circuit_TB();

parameter width = `width;
reg clk = 0, rst = 0, S = 0;
reg signed [width-1:0] A, B, C;
reg sign_A, sign_B, sign_C;
wire R;
wire signed [width-1:0] out, R1_out, R2_out;

circuit circuit1(clk, rst, S, A, B, C, R, out, R1_out, R2_out);

always begin
    #5 clk <= ~clk;
end

initial begin
    forever begin
        A = $urandom_range(100);
        B = $urandom_range(100);
        C = $urandom_range(10000);
        sign_A = $urandom_range(2);
        sign_B = $urandom_range(2);
        sign_C = $urandom_range(2);
        if(sign_A) A = -A;
        if(sign_B) B = -B;
        if(sign_C) C = -C;

        S = 1;
        wait(R);
        $display("\tA = %d, \tB = %d, \tC = %d, \tout = %d\n\n", A, B, C, out);

        rst = 1;
        #15 rst = 0;
    end
end

initial
    #10000 $finish;

endmodule
```

بخشی از خروجی test bench به صورت زیر است که گواهی بر صحت عملکرد مدار است.

```
A =       20, B =        2, C =    -8572, out =    -8572
A =       79, B =       79, C =       87, out =       87
A =       93, B =       21, C =     2315, out =     1953
A =       13, B =      -51, C =     9837, out =     -663
A =       58, B =        8, C =    -3897, out =    -3897
A =       49, B =      -58, C =      555, out =    -2842
A =       70, B =       22, C =     2264, out =     1540
A =      -92, B =        5, C =     3630, out =     -460
A =       58, B =       36, C =     7446, out =     2088
A =       75, B =      -43, C =     4053, out =    -3225
A =       55, B =       31, C =     7636, out =     1705
A =        8, B =      -88, C =     3863, out =     -704
A =       46, B =       27, C =     6537, out =     1242
A =      -53, B =      -45, C =     3119, out =     2385
A =       81, B =       11, C =    -2375, out =    -2375
A =       71, B =       39, C =     8557, out =     2769
A =       79, B =       35, C =    -7490, out =    -7490
```

```
A =  -20, B =   3, C = -5037, out = -5037
A =  -97, B =  -78, C =  9883, out =  7566
A =   95, B =  23, C =  -359, out =  -359
A =  -45, B =  38, C =  6811, out = -1710
A =  -19, B =  52, C =  2509, out =  -988
A =   29, B =   3, C =  -263, out =  -263
A =    0, B =  -34, C = -5586, out = -5586
A =   40, B =  -11, C = -9743, out = -9743
A =   74, B =  27, C =  9601, out =  1998
A =   37, B =  58, C =  -228, out =  -228
A =  -11, B =  61, C =  2466, out =  -671
A =   69, B =  65, C = -8111, out = -8111
A =   31, B =  18, C = -1789, out = -1789
A =  -42, B =  -16, C =  7935, out =  672
A =   23, B =  62, C = -5354, out = -5354
A =   56, B =  41, C = -7612, out = -7612
A =   12, B =  58, C =  6722, out =  696
```